Render Props Pattern

6.3 – Render Props Pattern

🎯 What Interviewers Actually Ask

Question	Level	Frequency
"What is the Render Props pattern?"	🟡🔴	★★★☆☆
"How does it compare to HOCs and hooks?"	🟡🔴	★★★★☆
"Give a practical example"	🟡🔴	★★★☆☆

The Pattern

Render Props is a technique where a component receives a function as a prop (or as `children`) and calls it to determine what to render. The component provides data or behavior, and the consuming code decides how to display it.

JSX Copy

```
// Share mouse position data through a render prop
function MouseTracker({ render }) {
  const [position, setPosition] = useState({ x: 0, y: 0 });

  useEffect(() => {
    const handleMove = (e) => setPosition({ x: e.clientX, y: e.clientY });
    window.addEventListener('mousemove', handleMove);
    return () => window.removeEventListener('mousemove', handleMove);
  }, []);

  return render(position);
}

// The consumer decides what to render with that data
function App() {
  return (
    <MouseTracker
      render={({ x, y }) => <p>Mouse is at ({x}, {y})</p>}
    />
  );
}
```

Using `children` as a function (common variation)

JSX

Copy

```
function MouseTracker({ children }) {
  const [position, setPosition] = useState({ x: 0, y: 0 });
  // ... same event listener logic ...
  return children(position);
}

// Cleaner JSX syntax
<MouseTracker>
  {({ x, y }) => <p>Mouse is at ({x}, {y})</p>}
</MouseTracker>
```

When It Was Useful

Before hooks, render props were the main alternative to HOCs for sharing behavior. Components like `<Query>` in Apollo Client v2 and `<Formik>` used this pattern extensively.

The Modern Alternative

Custom hooks replaced render props for most use cases. The logic stays the same, but the ergonomics are better.

JSX Copy

```jsx
// Render props (pre-hooks)
<MouseTracker>
  {({ x, y }) => <p>({x}, {y})</p>}
</MouseTracker>

// Custom hook (modern)
function App() {
  const { x, y } = useMouse();
  return <p>({x}, {y})</p>;
}
```

When Render Props Still Make Sense

1. **Library components** that need to provide flexible rendering to consumers (like virtualized list renderers).
2. **Component-level inversion of control** where a parent provides the rendering logic to a generic child.

Interview answer: "Render Props is a pattern where a component calls a function prop to determine what to render. The component provides data or behavior, and the consumer controls the visual output. It was a popular alternative to HOCs before hooks, used by libraries like Apollo and Formik. Today, custom hooks are preferred for most cases because they're simpler and don't create extra nesting. But render props still have a place in library APIs where you need to let consumers control the rendering of individual items."

QUICK CHECK — no marks, just for you.

1. In the Render Props pattern as described in the lesson, how is responsibility split between the component that holds the pattern and the code that consumes it?

- ○ The component supplies the data or behavior, and the consumer decides how it is displayed
- ○ The component and the consumer both render markup, which React then merges into one tree
- ○ The consumer supplies the data through props, and the component renders it with a fixed layout
- ○ The component decides the markup, and the consumer only supplies the initial data

Check answer

2. The lesson shows a common variation where MouseTracker is written as `function MouseTracker({ children }) { ... return children(position); }`. What must the consumer pass between the `<MouseTracker>` tags for this to work?

- ○ A component reference, which MouseTracker instantiates with the position
- ○ A function that takes the position and returns JSX
- ○ Nothing — with the children variation, MouseTracker renders the position itself
- ○ Ordinary JSX elements, which MouseTracker clones with the position injected as props

Check answer

3. ...ooks have replaced render props for most use cases. Which situation does it still identify as a good fit for render props?

- Sharing state between sibling components that sit far apart in the tree
- A library component that needs to let consumers control flexible rendering, such as a virtualized list renderer
- Components that must keep working in codebases where hooks are unavailable
- Any component that needs to subscribe to browser events like mousemove

Check answer

‹ Previous
Higher-Order Components (HOC)

Next ›
Compound Component Pattern

LEARN MORE

ARTICLES & DOCS

Render Props (docs)
reactjs.org

VIDEOS



When to use the render props pattern
YouTube



Render props made simple
YouTube



Render prop pattern in React
YouTube

Hand-picked references for this lesson.



React.js Interview Kit